PRACTICA 4 DE PROGRAMACION

CHECA TITO ANDERSON CLEVER	CI: 14154013	RU: 1900944	FECHA:02/09/26

1. Antes de empezar, ejecute el código mostrando el acta con las notas 85.0 y 92.0.

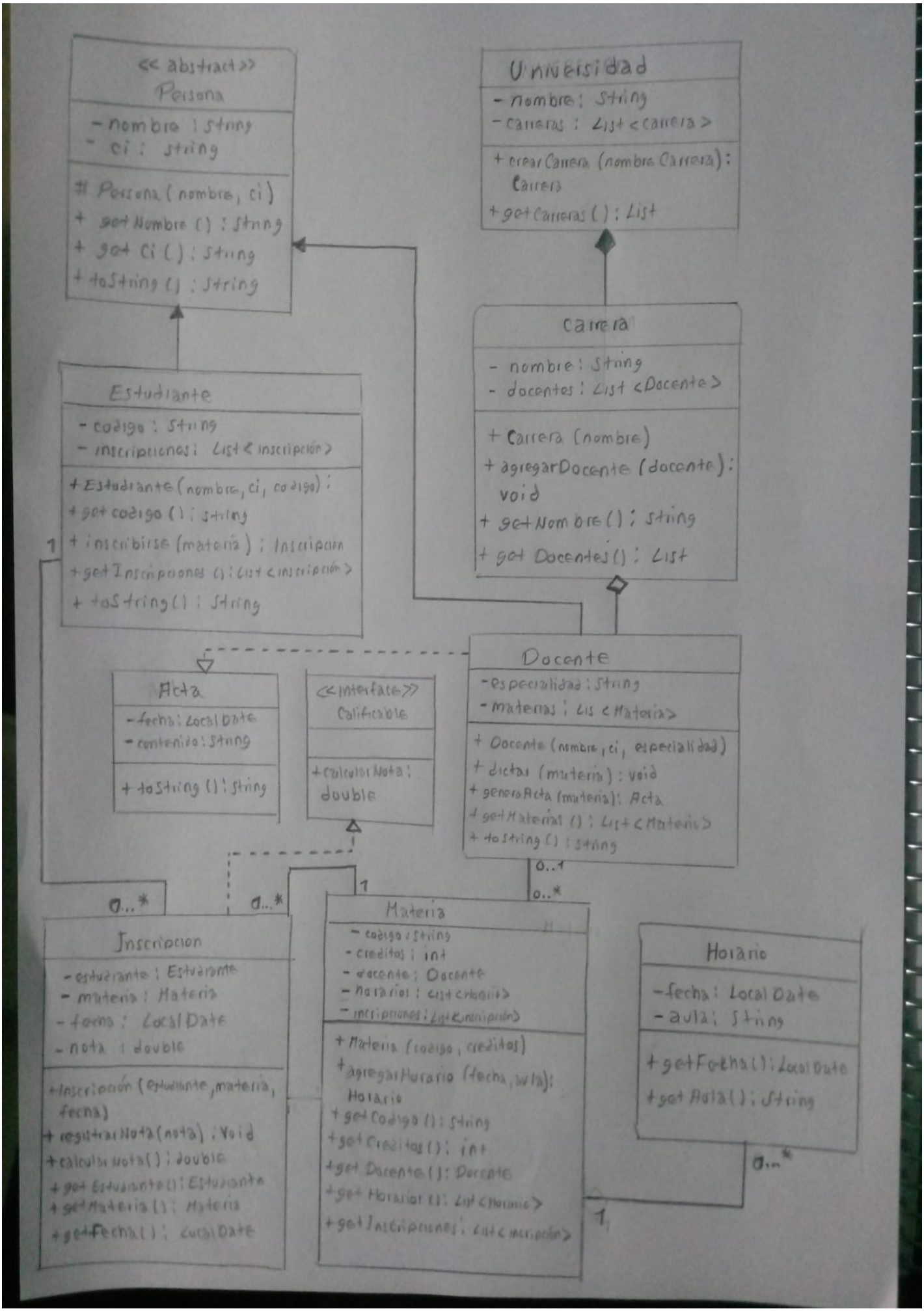
```
365
366         i1.registrarNota(85);
367         i2.registrarNota(92);
368
369         // HERENCIA + POLIMORFISMO
370
371         List<Persona> personas = Arrays.asList(docente, e1, e2);
372
373         personas.forEach(p -> System.out.println(p));
374
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS bash

```
anderson@andersonHP:~/Documentos/MATERIAS_UMSA/SEMESTREII/PROGRAMACION/LABORATORIS/4.LABORATORIO/PRACTICA_4$ java SistemaAc
ademico
Docente Ana Alba (Ingenieria de Software)
Estudiante Carlos Flores (codigo E-001)
Estudiante Maria Salas (codigo E-002)
Nota de Carlos Flores: 85.0
ACTA INF-121 - 2026-11-09
Carlos Flores : 85.0
Maria Salas : 92.0
```

Lic este el codigo ejecutado en la terminal

2. Dibuja a mano el diagrama de clases UML con multiplicidades y flechas correctas.


<< abstract >>
Persona
- nombre : String
- ci : string
Persona (nombre, ci)
+ getNombre () : String
+ get Ci () : String
+ toString () : String
Universidad
- nombre: String
- carreras : List <carrera>
+ crearCarrera (nombreCarrera): Carrera
+ getCarreras () : List
Carrera
- nombre: String
- docentes: List <Docente>
+ Carrera (nombre)
+ agregarDocente (docente): void
+ getNombre () ; string
+ get Docentes () : List
Estudiante
- codigo : String
- inscripciones: List < inscripción>
+ Estudiante (nombre, ci, codigo);
+ get codigo () ; string
+ inscribirse (materia) ; Inscripcion
+ getInscripciones () : List <inscripción>
+ toString () ; String
1
Docente
- especialidad : String
- materias ; Lis <Materia>
+ Docente (nombre, ci, especialidad)
+ dictar (materia) : void
+ generaActa (materia): Acta
+ getMaterias () ; List <Materia>
+ toString () ; string
Acta
- fecha: LocalDate
- contenido: String
+ toString () ; string
<<Interface>>
Calificable
+ calcularNota : double
0..*
0..*
1
0..*
Inscripcion
- estudiante : Estudiante
- materia : Materia
- fecha : LocalDate
- nota : double
+Inscripción (estudiante, materia, fecha)
+ registrarNota(nota) ; Void
+calcularNota() ; double
+ getEstudiante() : Estudiante
+ getMateria () ; Materia
+ getFecha () ; LocalDate
Materia
- codigo : String
- creditos ; int
- docente ; Docente
- horarios : List<Horario>
- inscripciones: List<inscripción>
+ Materia (codigo, creditos)
+ agregarHorario (fecha, aula): Horario
+ getCodigo () : String
+ getCreditos () ; int
+ get Docente () : Docente
+ get Horarios () : List <Horario>
+ getInscripciones : List <inscripción>
Horario
- fecha : LocalDate
- aula ; String
+ getFecha () : LocalDate
+ get Aula () ; String
0..*
1

3. Completa la tabla indicando el tipo de relación y la línea del código que lo demuestra.

Par de clases	Tipo de relación	Evidencia
Estudiante y Persona	Herencia	***class Estudiante extends Persona*** *que esta en la* (línea 31)
Universidad y Carrera	Composición	***Public Carrera crearCarrera(string nombreCarrera)*** crea una nueva Carrera y la guarda en la lista se encuentra en las (líneas 110-116)
Carrera y Docente	Agregación	***Public void agregarDocente(Docente docente)*** recibe un docente ya existente se encuentra en (líneas 138-140)
Materia y Horario	Composición	***Public Horario agregarHorario(LocalDate fecha ,String aula)*** crea un nuevo Horario y lo guarda , las lineas estan (líneas 167-173)
Docente y Materia	Asociación	***Public void dictar(Materia materia)*** registra la materia en el docente y al docente en la materia y estan en (líneas 77-80).
Inscripcion y Calificable	Implementación de interfaz	***class Inscripcion implements Calificable*** *se encuentra en* (línea 233)
Docente y Acta	Dependencia	***Public Acta generarActa(Materia materia)*** crea y devuelve un objeto Acta esto se encuentra en (líneas 84-86).

4. Explica por qué Docente solo depende de Acta y no se asocia. ¿Qué cambiaría en el código si se vuelve una asociación? (escriba parte del código).

Docente solamente depende de Acta porque utiliza esta clase temporalmente dentro del método **generarActa()**

```
public Acta generarActa(Materia materia) {

   return new Acta(materia);

}
```

El objeto Acta se crea se devuelve y no se guarda como atributo dentro de la clase Docente por eso es una dependencia y no una asociacion

¿Qué cambiaría si se convierte en asociación?

El docente tendría que guardar una referencia a las actas que genera

Lo priemero que agregamos lic es un atributo:

```
private final List<Acta> actas = new ArrayList<>();
```

Despues lic modificamos el metodo:

```
public Acta generarActa(Materia materia) {
    Acta acta = new Acta(materia);
    actas.add(acta);
    return acta;
}
```

Yo tambien agregaria un metodo para consultar las actas

```
public List<Acta> getActas() {
   return Collections.unmodifiableList(actas);
}
```

DIFERENCIAS:

- **Dependencia**: Docente utiliza Acta temporalmente
- **Asociacion:** Docente mantiene una referencia a las Actas que genera

5. ¿Qué diferencia hay entre crearCarrera() y agregarDocente() respecto a quién controla la parte? (Relación: Todo - Parte)

La diferencia esta en quien crea y quien controla la existencia del objeto

crearCarrera()

ES UNA COMPOSICION

```
public Carrera crearCarrera(String nombreCarrera) {
    Carrera d = new Carrera(nombreCarrera);
    carreras.add(d);
    return d;
}
```

- La Universidad crea la Carrera
- La Universidad administra las carreras que contiene
- La Carrera se considera una parte que depende de la Universidad

agregarDocente()

ES UNA AGREGACION

```
public void agregarDocente(Docente docente) {

    docentes.add(docente);

}
```

- La Carrera no crea al Docente
- El Docente ya existe antes de agregarse
- El Docente puede existir independientemente de la Carrera

DIFERENCIA QUE EXISTE ENTRE

crearCarrera()	*agregarDocente()*
Es una composicion y en una composicion el objeto principal controla la creacion y el ciclo de vida de sus partes	Es una agregacion y en una agregacion el objeto principal solamente reúne objetos que pueden existir independientemente

6. El código base funciona con datos correctos, pero acepta operaciones que dejan el modelo inconsistente; aquí se muestra algunos de ellos y se solicita corregirlo. Para cada caso, escribe el código (del main) que lo provoca, anota el resultado observado y propone una corrección (escriba parte del código).

CASO	**COMO PROVOCARLO**	**RESULTADO OBSERVADO**	**CORRECCION PROPUESTA**
Doble inscripción	Inscripcion i1 = e1.inscribirse(poo); Inscripcion i2 = e1.inscribirse(poo);	El estudiante queda inscrito dos veces en la misma materia se crean dos objetos `Inscripcion` y ambos se guardan en las listas	Modificar el metodo `inscribirse()` de la clase `Estudiante`: **public Inscripcion inscribirse(Materia materia) {** **for (Inscripcion i : inscripciones) {** **if (i.getMateria() == materia) {** **throw new IllegalArgumentException("El estudiante ya esta inscrito en esta materia");** **}** **}** **Inscripcion inscripcion = new Inscripcion(this, materia, LocalDate.now());** **inscripciones.add(inscripcion);** **materia.registrarInscripcion(inscripcion);** **return inscripcion;** **}** Ahora se comprueba si el estudiante ya tiene una inscripcion en esa materia antes de crear otra

Dos docentes en un curso	d1.dictar(poo) y luego d2.dictar(poo)	La materia termina teniendo como docente al segundo docente, porque `setDocente()` reemplaza la referencia anterior sin embargo ambos docentes conservan la materia en sus listas esto genera una inconsistencia	Modificar el método `dictar()` de la clase `Docente` **public void dictar(Materia materia) {** **if (materia.getDocente() != null) {** **throw new IllegalArgumentException(** **"La materia ya tiene un docente"** **);** **}** **materias.add(materia);** **materia.setDocente(this);** **}** pues con esta modificacion una materia no puede tener dos docentes asignados mediante este metodo
Nota sin registrar	Consultar calcularNota() antes de registrarNota()	me muestra esto 0.0 a lo que entendi esto sucede porque el atributo `double nota` tiene inicialmente el valor predeterminado `0.0` en Java el problema es que no podemos distinguir entre una nota realmente registrada de cero y una nota que todavia no fue registrada	Cambiar el atributo: **private Double nota;** Modificar el metodo **public void registrarNota(double nota) {** **if (nota < 0 || nota > 100) {** **throw new IllegalArgumentException(** **"La nota debe estar entre 0 y 100"** **);** **}** **this.nota = nota;** **}** **Modificar `calcularNota()`** **@Override** **public double calcularNota() {** **if (nota == null) {** **throw new IllegalStateException(** **"La nota todavia no fue registrada"** **);** **}** **return nota;** **}** Ahora si no se registra una nota el programa informa que todavia no existe una calificacion

Docente en dos carreras	agregarDocente() del mismo docente en dos carreras	El mismo docente queda registrado en dos carreras diferentes esto puede generar inconsistencias si el modelo establece que cada docente solamente puede pertenecer a una carrera	Agregar un atributo a la clase `Docente` **private Carrera carrera;** Agregar un método para asignar la carrera **void asignarCarrera(Carrera carrera) {** **if (this.carrera != null && this.carrera != carrera) {** **throw new IllegalArgumentException(** **"El docente ya pertenece a otra carrera"** **);** **}** **this.carrera = carrera;** **}** **Modificar el método `agregarDocente()` de `Carrera`** **public void agregarDocente(Docente docente) {** **docente.asignarCarrera(this);** **docentes.add(docente);** **}** Ahora el docente solamente puede pertenecer a una carrera
Composición no estricta	new Sesion(...) desde el main	Es posible crear un objeto `Horario` directamente desde el `main` sin utilizar el método `agregarHorario()` de `Materia` esto debilita la composicion porque la creacion de las partes no esta restringida al objeto que las contiene	Una alternativa es convertir `Horario` en una clase interna estática de `Materia` y utilizar un constructor privado Dentro de `Materia hacemos:` **public static class Horario {** **private final LocalDate fecha;** **private final String aula;** **private Horario(LocalDate fecha, String aula) {** **this.fecha = fecha;** **this.aula = aula;** **}** **}** **El método de creación dentro de `Materia`** **public Horario agregarHorario(LocalDate fecha, String aula) {** **Horario h = new Horario(fecha, aula);** **horarios.add(h);** **return h;** **}** Asi el constructor privado solamente puede utilizarse desde la propia clase contenedora `Materia`

7. Cada ampliación exige elegir primero la relación correcta y justificarla. Agrega a Ayudante que es estudiante y colabora con un docente ¿Será una Herencia, Dependencia, Asociación, Composición o Agregación? Actualiza el diagrama UML con la clase nueva y sus multiplicidades.

Ayudante y Estudiante:

Un ayudante es un estudiante por lo tanto puede **heredar** de la clase Estudiante entonces podemos hacer una herencia

```
class Ayudante extends Estudiante {
    public Ayudante(String nombre,String ci,String codigo ) {
        super(nombre, ci, codigo); }
}
```

Ayudante y Docente

El ayudante colabora con un docente por lo tanto necesita mantener una referencia a ese docente entonces podemos hacer una asociacion

Modificamos la clase Ayudante:

```
class Ayudante extends Estudiante {
    private Docente docente;
    public Ayudante(String nombre,String ci,String codigo,Docente docente) {
        super(nombre, ci, codigo);
        this.docente = docente;
    }
    public Docente getDocente() {
        return docente;}
}
```

Modificar la clase Docente

Para que el docente tambien conozca a sus ayudantes agregamos una lista

```
private final List<Ayudante> ayudantes = new ArrayList<>();
```

Agregamos el metodo

```
public void agregarAyudante(Ayudante ayudante) {
    if (!ayudantes.contains(ayudante)) {
        ayudantes.add(ayudante);
    }
}
```

Ejemplo de utilización en el main

```
Docente docente = new Docente("Ana Alba","1234567","Ingenieria de Software");
Ayudante ayudante = new Ayudante("Carlos Flores", "7654321", "E-001", docente);
docente.agregarAyudante(ayudante);
```

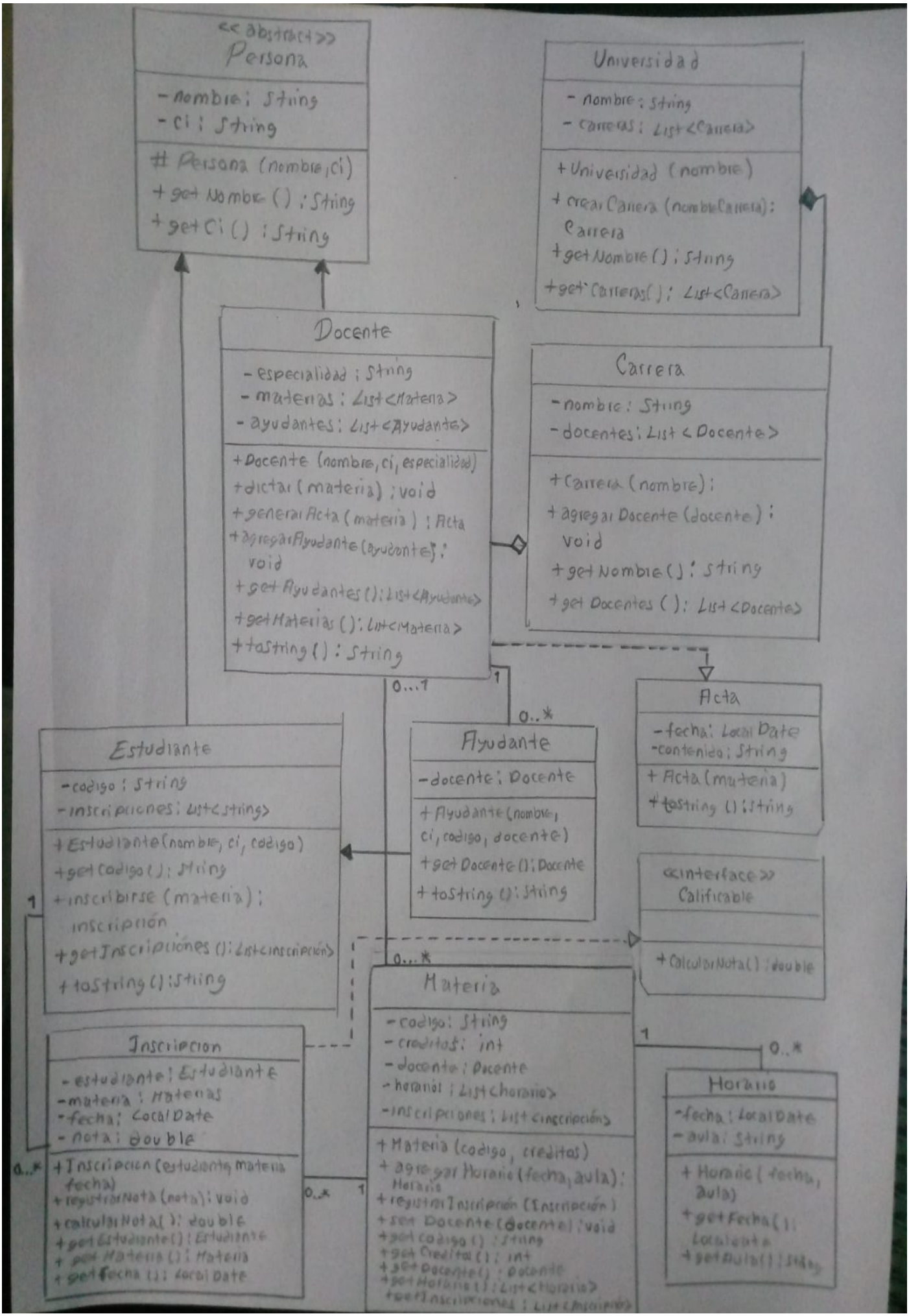
SUS RELACIONES CORRECTAS SON

PAR DE CLASES	RELACIONES
Ayudante y Estudiante	Herencia
Ayudante y Docente	Asociación

JUSTIFICACION

- **Herencia**: Ayudante es un tipo de Estudiante y hereda sus atributos y metodos
- **Asociación**: Ayudante mantiene una referencia al Docente con quien colabora

UML DEL CODIGO MODIFICADO


<<abstract>>
Persona
- nombre: String
- ci: String
Persona(nombre, ci)
+ getNombre(): String
+ getCi(): String
Universidad
- nombre: String
- carreras: List<Carrera>
+ Universidad(nombre)
+ crearCarrera(nombreCarrera): Carrera
+ getNombre(): String
+ getCarreras(): List<Carrera>
Docente
- especialidad: String
- materias: List<Materia>
- ayudantes: List<Ayudante>
+ Docente(nombre, ci, especialidad)
+ dictar(materia): void
+ generarActa(materia): Acta
+ agregarAyudante(ayudante): void
+ getAyudantes(): List<Ayudante>
+ getMaterias(): List<Materia>
+ toString(): String
Carrera
- nombre: String
- docentes: List<Docente>
+ Carrera(nombre)
+ agregarDocente(docente): void
+ getNombre(): String
+ getDocentes(): List<Docente>
Acta
- fecha: LocalDate
- contenido: String
+ Acta(materia)
+ toString(): String
Estudiante
- codigo: String
- inscripciones: List<String>
+ Estudiante(nombre, ci, codigo)
+ getCodigo(): String
+ inscribirse(materia): inscripción
+ getInscripciones(): List<Inscripción>
+ toString(): String
Ayudante
- docente: Docente
+ Ayudante(nombre, ci, codigo, docente)
+ getDocente(): Docente
+ toString(): String
<<interface>>
Calificable
+ calcularNota(): double
Materia
- codigo: String
- creditos: int
- docente: Docente
- horarios: List<Horario>
- inscripciones: List<Inscripción>
+ Materia(codigo, creditos)
+ agregarHorario(fecha, aula): Horario
+ registrarInscripción(Inscripción)
+ setDocente(docente): void
+ getCodigo(): String
+ getCreditos(): int
+ getDocente(): Docente
+ getHorarios(): List<Horario>
+ getInscripciones(): List<Inscripción>
Inscripcion
- estudiante: Estudiante
- materia: Materias
- fecha: LocalDate
- nota: double
+ Inscripcion(estudiante, materia, fecha)
+ registrarNota(nota): void
+ calcularNota(): double
+ getEstudiante(): Estudiante
+ getMateria(): Materia
+ getFecha(): LocalDate
Horario
- fecha: LocalDate
- aula: String
+ Horario(fecha, aula)
+ getFecha(): LocalDate
+ getAula(): String
0...1
1
0..*
1
1
0..*
1
0..*
0..*
1
1
0..*
0..*
1